ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ · ЛЕКЦИЯ 5

Основы метода Монте-Карло

Случайная выборка, интегрирование и случайное блуждание

Лю Шисян

Кафедра теплофизики (Э6)
МГТУ им. Н.Э. Баумана
Москва / 2026

Место этой лекции в курсе



Лекция 5 — Основы метода Монте-Карло

Метод Монте-Карло использует случайную выборку и статистическое усреднение для вычисления детерминированных величин.

Задачи занятия:

- генерировать и анализировать случайные выборки;
- вычислять интегралы методом Монте-Карло;
- объяснять, как точность Monte Carlo зависит от числа выборок;
- объяснять связь случайного блуждания и диффузии.

План

1. Основы метода Монте-Карло
2. Интегрирование методом Монте-Карло
3. Случайное блуждание и диффузия
4. Практика и итоги

1

Основы метода Монте-Карло

Краткая история метода Монте-Карло



Игла Бюффона
случайный эксперимент для оценки математической величины

Los Alamos
S. Ulam, J. von Neumann
случайное моделирование физических процессов

Metropolis & Ulam
статья
“The Monte Carlo Method”

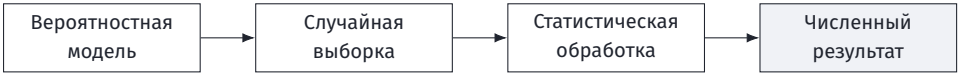
Название метода

Название связано с казино Монте-Карло в Монако.

Метод Монте-Карло

Определение

Методы Монте-Карло — это класс численных методов, в которых искомая величина оценивается по результатам **случайной выборки**.



Случайная величина и распределение

Случайная величина

Случайная величина X — числовая величина, значение которой определяется исходом случайного эксперимента.

Примеры:

- результат броска кубика;
- координата случайной точки;
- время между случайными событиями;
- длина свободного пробега частицы.

Распределение

Распределение описывает, **какие значения может принимать** X и насколько они вероятны. Для непрерывной величины используется плотность вероятности $p_X(x)$.
Вероятность попасть в интервал:

$$P(a < X < b) = \int_a^b p_X(x)\,dx.$$

Примеры записи:

$$X \sim U(a, b), \qquad X \sim \mathcal{N}(\mu, \sigma^2).$$

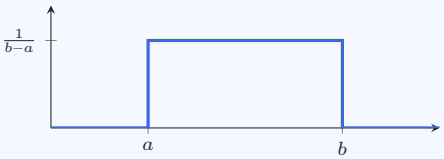
Основные распределения

Равномерное распределение

$$X \sim U(a, b)$$

Плотность вероятности:

$$p_X(x) = \frac{1}{b-a}, \qquad a \le x \le b.$$



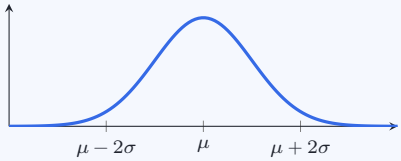
Все интервалы одинаковой длины внутри $[a, b]$ имеют одинаковую вероятность.

Нормальное распределение

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

Плотность вероятности:

$$p_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right].$$



μ задаёт центр распределения, а σ — характерный разброс.

Математическое ожидание и дисперсия

Математическое ожидание

Среднее значение случайной величины:

$$\mu = \mathbb{E}[X]$$

Для непрерывной величины:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x\, p_X(x)\, dx.$$

В Monte Carlo оно оценивается по выборочному среднему:

$$\bar{X} = \frac{1}{N} \sum_{i=1}^{N} X_i$$

Дисперсия

Разброс относительно среднего:

$$\sigma^2 = \mathrm{Var}(X) = \mathbb{E}[(X - \mu)^2]$$

Стандартное отклонение:

$$\sigma = \sqrt{\mathrm{Var}(X)}$$

Чем больше σ, тем шире разброс значений около μ.

Псевдослучайные числа в NumPy

```
rng = np.random.default_rng(42)
u = rng.uniform(0, 1, 1000)
z = rng.normal(0, 1, 1000)
i = rng.integers(0, 10, 1000)
```

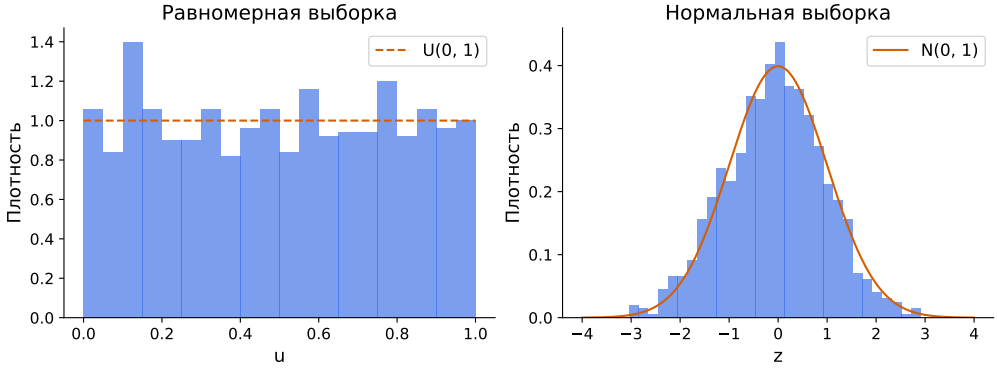
- `uniform`: числа от 0 до 1;
- `normal`: среднее 0, стандартное отклонение 1;
- `integers`: целые числа от 0 до 9.

Seed и последовательность

Одинаковые генератор, seed и порядок вызовов воспроизводят результат.

Повторный вызов того же генератора продолжает последовательность.

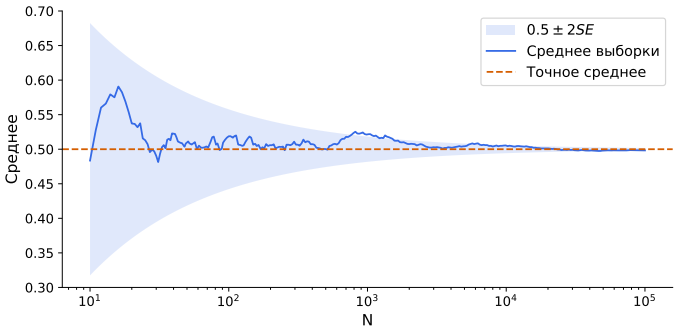
Случайная выборка и плотность вероятности



По 1000 значений. Площадь каждой гистограммы равна единице.

Столбцы колеблются около теоретической плотности.

Выборочное среднее



Равномерная выборка

$$X_i \sim U(0,1), \quad \mathbb{E}[X] = 0.5.$$

В каждом следующем среднем используем больше значений той же выборки.

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^{N} X_i.$$

Полоса: примерно две стандартные ошибки при каждом N.

Оценка числа π по площади круга

Рассмотрим единичный квадрат и четверть единичного круга.

Площадь квадрата:

$$S_{\square} = 1.$$

Площадь четверти круга:

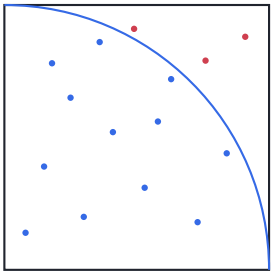
$$S_{\circ} = \frac{\pi}{4}.$$

Если точки равномерно распределены в квадрате, то

$$\frac{N_{\text{in}}}{N} \approx \frac{\pi}{4}.$$

Отсюда

$$\boxed{\pi \approx 4\frac{N_{\text{in}}}{N}}.$$



$x^2 + y^2 \leq 1$

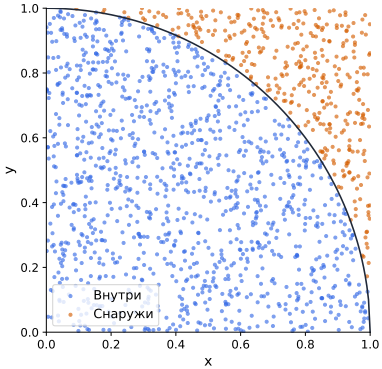
Оценка числа π на Python

```
import numpy as np

rng = np.random.default_rng(42)
N = 100_000
x = rng.random(N)
y = rng.random(N)
inside = x**2 + y**2 <= 1
pi_mc = 4 * inside.mean()
```

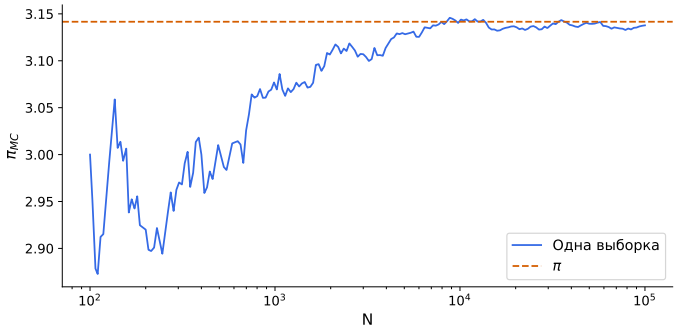
`inside.mean()` даёт долю попаданий.

Результат: `3.13776`



Показаны первые 1500 из 100 000 точек.

Одна выборка: оценка числа π



Для каждого N берём первые N пар координат.

Оценка колеблется около π.

Ошибка отдельной реализации не обязана уменьшаться при каждом увеличении N.

Стандартная ошибка оценки

```
p_hat = inside.mean()
se_pi = 4 * np.sqrt(
    p_hat * (1 - p_hat) / N
)
ci_pi = (pi_mc - 1.96 * se_pi,
         pi_mc + 1.96 * se_pi)
```

При $N = 100\,000$**:**

$$SE \approx 0.00520.$$

$$\pi_{MC} \pm 1.96 SE \approx [3.12757,\ 3.14795].$$

Что означает интервал

При многократном повторении опыта примерно 95% таких интервалов содержат π.

Это приближение для большого числа независимых точек.

Независимые повторы оценки π

```
def estimate_pi(n, rng):
    x = rng.random(n)
    y = rng.random(n)
    return 4 * np.mean(x**2 + y**2 <= 1)

rng_repeats = np.random.default_rng(2026)
sample_sizes = np.array([
    100, 1000, 10_000, 100_000, 1_000_000
])
R = 30
estimates = np.empty((sample_sizes.size, R))
for j, n in enumerate(sample_sizes):
    for r in range(R):
        estimates[j, r] = estimate_pi(n, rng_repeats)
```

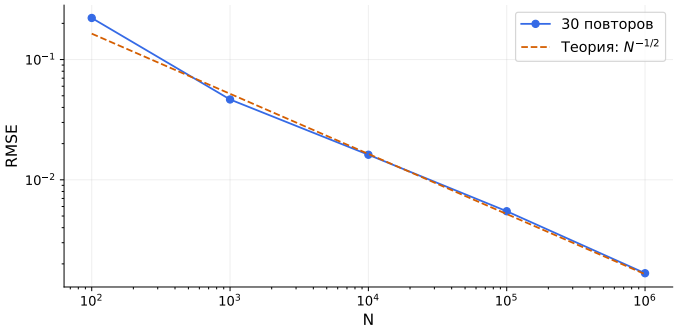
30 опытов для каждого N

Генератор создаём один раз перед циклами.

Каждый вызов получает новую порцию случайных чисел.

`estimates[j, r]` хранит результат одного опыта.

Как уменьшается статистическая ошибка



Ошибка по 30 опытам

$$\text{RMSE} = \sqrt{\frac{1}{R}\sum_{r=1}^{R}(\pi_r - \pi)^2}.$$

Для независимой выборки:

$$\text{RMSE} \propto N^{-1/2}.$$

В 10 раз меньшая ошибка требует примерно в 100 раз больше точек.

2

Интегрирование методом Монте-Карло

Оценка интеграла по случайной выборке

Рассмотрим интеграл

$$I = \int_a^b f(x)\,dx.$$

Пусть

$$X \sim U(a,b), \qquad p_X(x) = \frac{1}{b-a}.$$

Тогда

$$\mathbb{E}[f(X)] = \int_a^b f(x) p_X(x)\,dx$$

и, следовательно,

$$I = (b-a)\mathbb{E}[f(X)]$$

В Monte Carlo математическое ожидание заменяем выборочным средним:

$$\mathbb{E}[f(X)] \approx \frac{1}{N}\sum_{i=1}^{N} f(x_i),$$

где

$$x_i \sim U(a,b).$$

Получаем:

$$I_N = (b-a)\frac{1}{N}\sum_{i=1}^{N} f(x_i)$$

Интеграл функции e^{-x^2}

Рассмотрим интеграл

$$I = \int_0^1 e^{-x^2}\, dx.$$

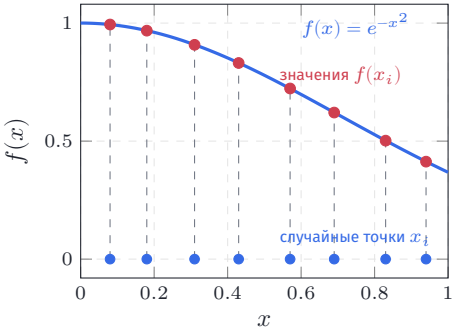
Шаг 1. Случайно выбираем точки

$$x_1, x_2, \dots, x_N \sim U(0,1).$$

Шаг 2. Вычисляем $f(x_i) = e^{-x_i^2}$.

Шаг 3. Усредняем эти значения:

$$I_N = \frac{1}{N} \sum_{i=1}^{N} e^{-x_i^2}.$$



Длина интервала: $b - a = 1$.

Интегрирование на Python

```
from scipy.special import erf

rng = np.random.default_rng(42)
N = 100_000
a, b = 0.0, 1.0
x = rng.uniform(a, b, N)
f = np.exp(-x**2)
I_mc = (b - a) * f.mean()
SE = (b - a) * f.std(ddof=1) / np.sqrt(N)
I_exact = np.sqrt(np.pi) * erf(1) / 2
```

Интеграл на $[0, 1]$

$$I_{MC} = 0.74639593.$$

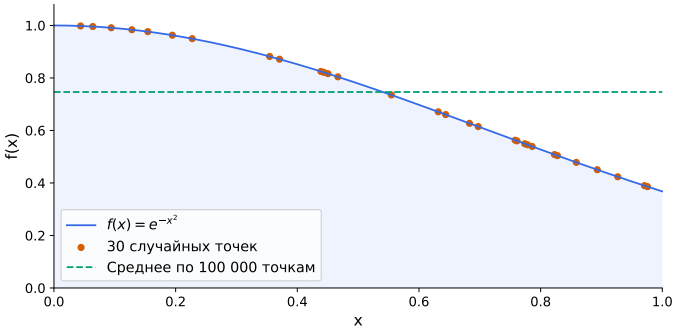
$$SE = 0.00063516.$$

Эталон для проверки

$$I_{\text{ref}} = 0.74682413.$$

`ddof=1`: выборочная дисперсия с делителем $N - 1$.

Выборка и среднее значение функции

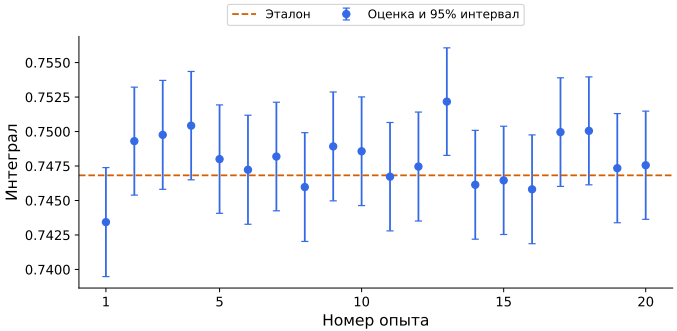


Показаны первые 30 случайных точек.

Среднее рассчитано по всем 100 000 значениям функции.

Длина интервала равна 1, поэтому это среднее и есть оценка интеграла.

Повторные оценки интеграла



20 независимых опытов

В каждом опыте 10 000 точек.

Полосы: $I_{MC} \pm 1.96SE$.

В этом расчёте эталон попал в 19 из 20 интервалов.

Для другого seed число может измениться.

Монте-Карло в высокой размерности

Для равномерных независимых точек $\mathbf{x}_i \in \Omega$:

$$I = \int_\Omega f(\mathbf{x})\, d\mathbf{x}, \qquad I_N = \frac{V_\Omega}{N} \sum_{i=1}^{N} f(\mathbf{x}_i).$$

Регулярная сетка

m точек по каждой координате:

$$N_{\text{grid}} = m^d.$$

Для $m = 100$, $d = 6$ нужно 10^{12} точек. Это проявление **проклятия размерности**.

Monte Carlo

При конечной дисперсии

$$\text{RMSE}(I_N) = \frac{V_\Omega \sigma_f}{\sqrt{N}}.$$

Показатель $-1/2$ не зависит от d, но σ_f и стоимость одного вычисления могут зависеть от размерности.

Шестимерный интеграл на Python

$$I = \int_{[0,1]^6} \exp\left(-\sum_{j=1}^{6} x_j^2\right) d\mathbf{x}.$$

```
rng6 = np.random.default_rng(42)
N6, d = 200_000, 6
x6 = rng6.random((N6, d))
f6 = np.exp(-np.sum(x6**2, axis=1))
I6_mc = f6.mean()
SE6 = f6.std(ddof=1) / np.sqrt(N6)
I6_exact = I_exact**d
```

x6.shape: (200000, 6)
f6.shape: (200000,)

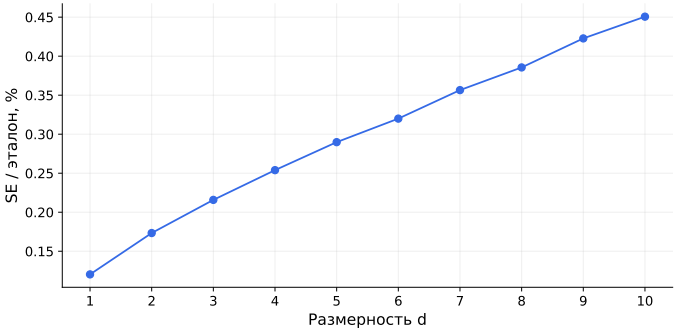
Одна строка = одна точка

`axis=1` суммирует квадраты шести координат.

$I_{MC} = 0.17359944$
$SE = 0.00028016$

$I_{\text{ref}} = 0.17350423$

Размерность и относительная ошибка



По 50 000 точек

Одна и та же функция в размерностях от 1 до 10.

Относительная стандартная ошибка растёт с размерностью.

У этого примера есть эталон: произведение одномерных интегралов.

3

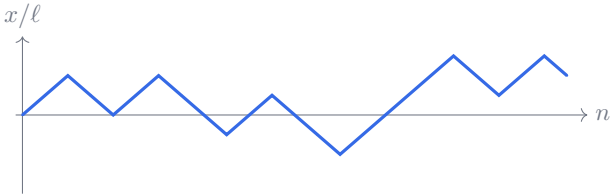
Случайное блуждание и диффузия

Одномерное случайное блуждание

Через каждый интервал времени Δt частица делает случайный шаг длины ℓ:

$$x_{n+1} = x_n + \Delta x_n,$$

где $\Delta x_n = \begin{cases} +\ell, & P = 1/2, \\ -\ell, & P = 1/2. \end{cases}$



После N шагов:

$$x_N = \Delta x_1 + \Delta x_2 + \cdots + \Delta x_N = \sum_{i=1}^{N} \Delta x_i$$

Среднее и среднеквадратичное смещение

После N шагов

$$x_N = \sum_{i=1}^{N} \Delta x_i.$$

Среднее положение

Из-за симметрии шагов

$$\langle \Delta x_i \rangle = \frac{1}{2}\ell + \frac{1}{2}(-\ell) = 0.$$

Поэтому

$$\langle x_N \rangle = 0$$

Среднее положение ансамбля остаётся в начале координат.

Среднеквадратичное смещение

Для независимых шагов

$$\langle (\Delta x_i)^2 \rangle = \ell^2,$$

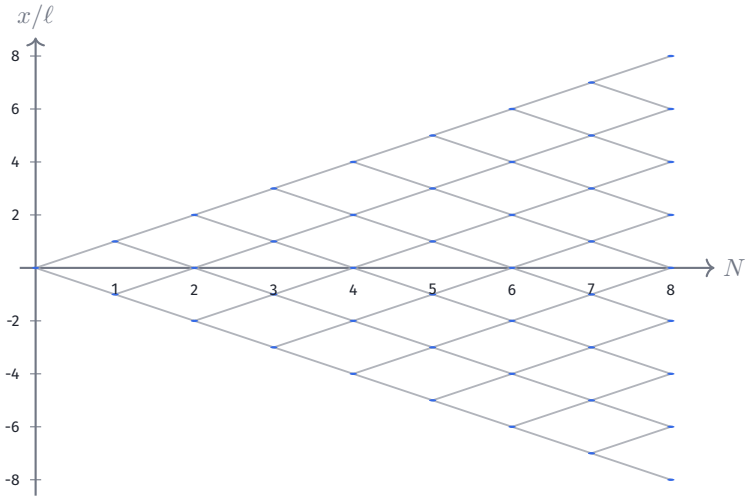
Поэтому

$$\langle x_N^2 \rangle = N\ell^2$$

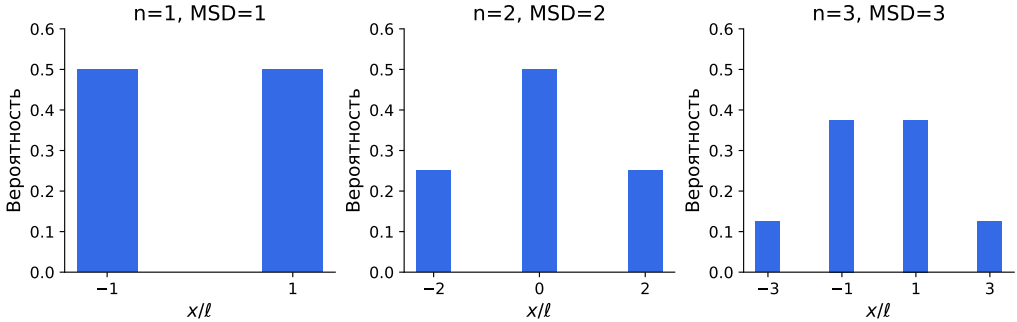
и характерное расстояние

$$x_{\text{rms}} = \sqrt{\langle x_N^2 \rangle} = \ell\sqrt{N}$$

Все возможные траектории случайного блуждания



Первые три шага: распределение и MSD



Перебраны все 2^n равновероятных траекторий при $\ell = 1$.

Несколько траекторий могут закончиться в одной точке.

Коэффициент диффузии случайного блуждания

Для одномерного случайного блуждания $\langle x^2 \rangle = N\ell^2$. Каждый шаг занимает время Δt, поэтому

$$t = N\Delta t.$$

Следовательно,

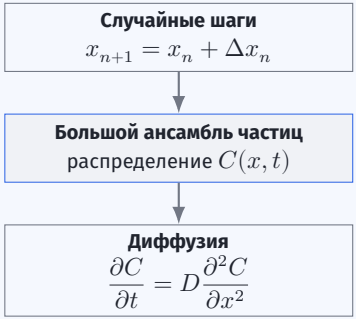
$$\langle x^2 \rangle = \frac{\ell^2}{\Delta t}\, t.$$

Для одномерной диффузии

$$\langle x^2 \rangle = 2Dt$$

Отсюда

$$D = \frac{\ell^2}{2\Delta t}$$



Непрерывный предел случайного блуждания

Пусть $P(x,t)$ — вероятность найти частицу в положении x в момент времени t.
Чтобы после следующего шага оказаться в точке x, частица должна прийти либо слева, либо справа:

$$\boxed{P(x,t+\Delta t) = \frac{1}{2}P(x-\ell,t) + \frac{1}{2}P(x+\ell,t)}$$



При $\ell, \Delta t \to 0$ и фиксированном $D = \ell^2/(2\Delta t)$ получаем уравнение для непрерывной плотности:

$$\boxed{\frac{\partial P}{\partial t} = D\frac{\partial^2 P}{\partial x^2}}, \qquad \boxed{D = \frac{\ell^2}{2\Delta t}}$$

Ансамбль случайных блужданий на Python

```
rng = np.random.default_rng(42)
M, N = 20_000, 500
ell, dt = 1.0, 1.0
steps = rng.choice([-ell, ell], size=(M, N))
x = np.cumsum(steps, axis=1)
t = np.arange(1, N + 1) * dt
mean_x = x.mean(axis=0)
msd = np.mean(x**2, axis=0)
D = ell**2 / (2 * dt)
```

Массив $20\,000 \times 500$

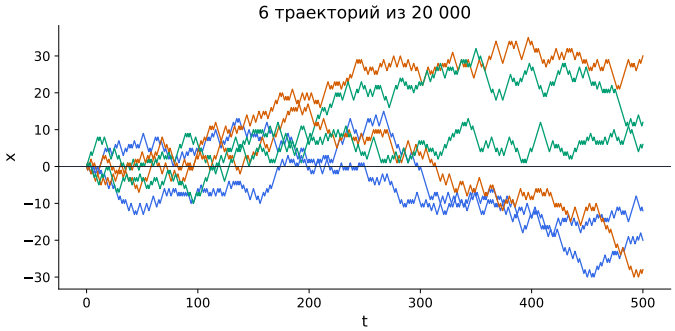
Строка: траектория одной частицы.

Столбец: положения всех частиц в один момент времени.

`axis=1`: складываем шаги.

`axis=0`: усредняем по частицам.

Отдельные траектории

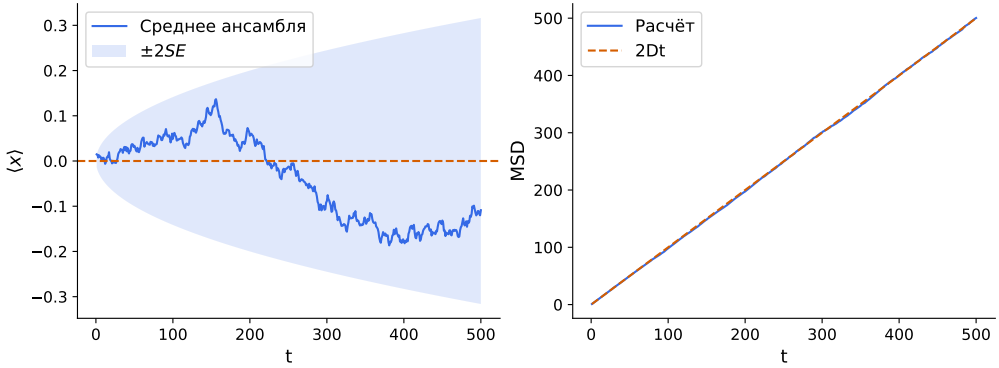


Все частицы начинают в нуле.

На каждом шаге выбирают $+1$ или -1 независимо.

Длина и время заданы в условных единицах.

Среднее положение и MSD ансамбля



20 000 частиц, 500 шагов. Теоретически $\langle x \rangle = 0$, $\langle x^2 \rangle = 2Dt$.

При $t = 500$: $\langle x \rangle = -0.1088$, MSD $= 500.3224$.

Коэффициент диффузии из расчёта

```
final_x = x[:, -1]
D_mc = np.mean(final_x**2) / (2 * t[-1])
SE_D = np.std(final_x**2, ddof=1) / (
    2 * t[-1] * np.sqrt(M)
)
```

$$D_{MC} = 0.50032, \qquad SE_D = 0.00503.$$

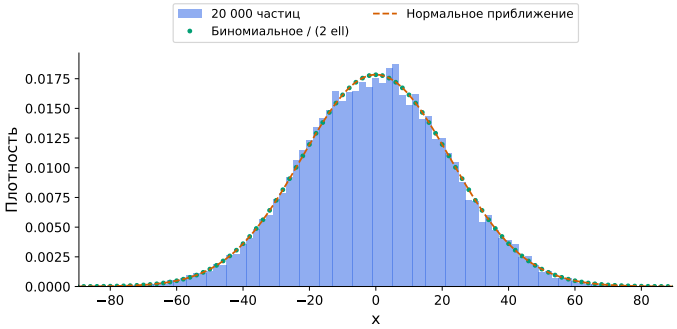
Теория

$$D = \frac{\ell^2}{2\Delta t} = 0.5.$$

Используем конечные положения независимых частиц.

Точки MSD в соседние моменты времени коррелируют.

Распределение положений после 500 шагов



Доступны только чётные x/ℓ

Ширина столбца: 2ℓ.

Точки показывают точные биномиальные вероятности, делённые на 2ℓ.

Пунктир: нормальное приближение с $\sigma = \ell\sqrt{N}$.

4

Практика и итоги

Типовая структура алгоритма Монте-Карло



Практическое задание

Оценка π

- Увеличьте число независимых повторов с 30 до 100.
- Сравните RMSE и ошибку одной реализации.

Интегрирование

- Увеличьте число точек в четыре раза. Как изменилась SE?

Случайное блуждание

- Удвойте длину шага при прежнем Δt.
- Сравните новый MSD и коэффициент диффузии с теорией.

Итоги лекции

- Генератор создаёт воспроизводимую случайную выборку.
- Среднее по выборке даёт оценку интеграла.
- Для независимых выборок с конечной дисперсией типичная ошибка убывает как $N^{-1/2}$.
- Независимые симметричные шаги приводят к диффузии: MSD $= 2Dt$.

github.com/lyushisyan/computational-methods-course